

Universidade de Pernambuco
Professor: Emerson Remigio da Silva
Disciplina: Computação Gráfica
Discentes: Gabriel Gomes Barbosa,
Leonardo Marcelo S. Da Paz

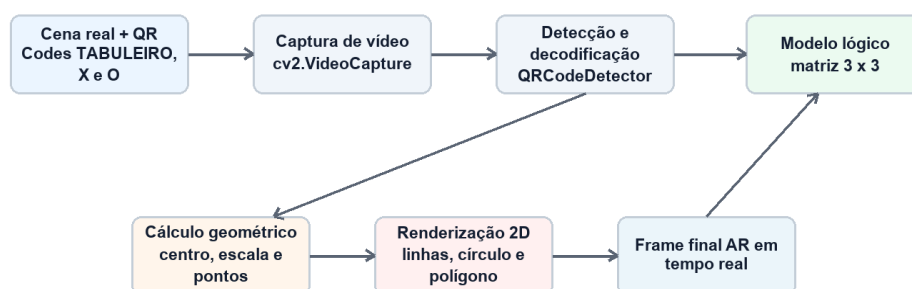
Modelagem e Fundamentação Teórica: Jogo da Velha com Realidade Aumentada

1. Visão Geral da Solução

O projeto consiste em uma aplicação de Realidade Aumentada baseada em vídeo (video-based AR), desenvolvida em Python com a biblioteca OpenCV, que viabiliza a execução de um Jogo da Velha virtual projetado sobre o ambiente real capturado por uma câmera. O sistema adota QR Codes como marcadores fiduciais, onde a tag "TABULEIRO" estabelece a matriz geométrica de projeção e as tags "X" e "O" identificam o posicionamento das peças virtuais. Enquanto a primeira etapa validou a viabilidade técnica do rastreamento de múltiplos marcadores e da renderização de primitivas em tempo real, esta segunda entrega foca na estruturação arquitetural e modelagem lógica do sistema. Propõe-se aqui a transição do protótipo puramente visual para um sistema dotado de estado interno controlado (matriz3x3), detalhando o pipeline de execução e os modelos de dados necessários para as próximas fases de desenvolvimento.

2. Arquitetura Geral do Sistema

A arquitetura estrutural do sistema foi projetada de forma modular, estabelecendo uma clara separação de responsabilidades entre a captura dos fluxos de entrada, o processamento de visão computacional, os cálculos geométricos espaciais, a máquina de estados lógico do jogo e a renderização gráfica final. Essa abordagem desacoplada simplifica o isolamento de componentes em testes, otimiza os procedimentos de manutenção corretiva e viabiliza a evolução incremental do sistema para a implementação das regras complexas da partida.

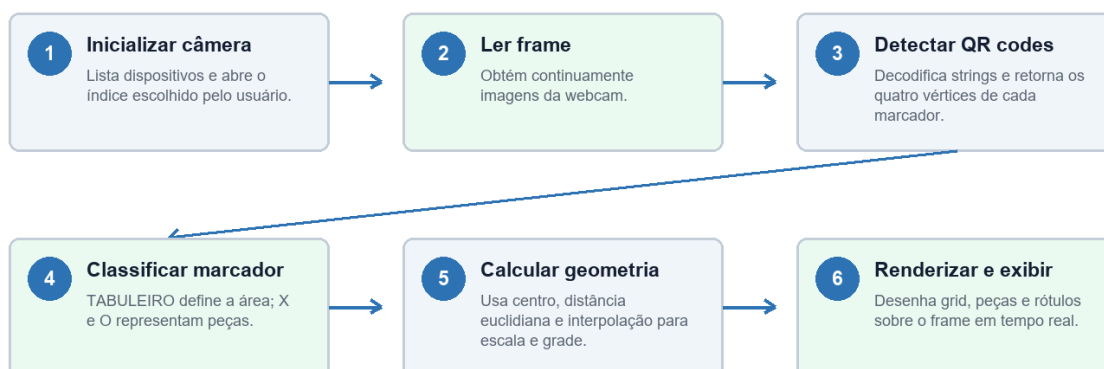


Fluxo principal: capturar frame -> localizar marcadores -> calcular coordenadas -> atualizar estado -> desenhar overlays -> exibir resultado.

Módulo	Responsabilidade	Tecnologias/artefatos
Entrada de vídeo	Selecionar câmera disponível e capturar frames continuamente.	cv2.VideoCapture, webcam/celular como webcam
Deteção de marcadores	Localizar QR Codes no frame, decodificar o texto e obter os quatro vértices do marcador.	cv2.QRCodeDetector.de tectAndDecodeMulti
Geometria e mapeamento	Calcular centro, largura, altura, escala, interpolação dos pontos da grade e limites do tabuleiro.	NumPy, normas vetoriais, coordenadas 2D
Modelo lógico do jogo	Representar o tabuleiro como matriz 3 x 3, associar peças às células e validar estados.	Matriz de estados, regras do jogo da velha
Renderização AR	Desenhar linhas do tabuleiro, peças X/O, contorno dos marcadores e textos de apoio sobre o frame.	cv2.line, cv2.circle, cv2.polylines, cv2.putText
Saída e interação	Exibir o frame aumentado em tempo real e permitir encerramento da aplicação.	cv2.imshow, cv2.waitKey

3. Etapas de Funcionamento

O funcionamento do sistema ocorre em um laço contínuo de processamento de imagem. A cada frame capturado, o sistema detecta os marcadores presentes, interpreta seu conteúdo textual e desenha elementos virtuais sobre as posições correspondentes.



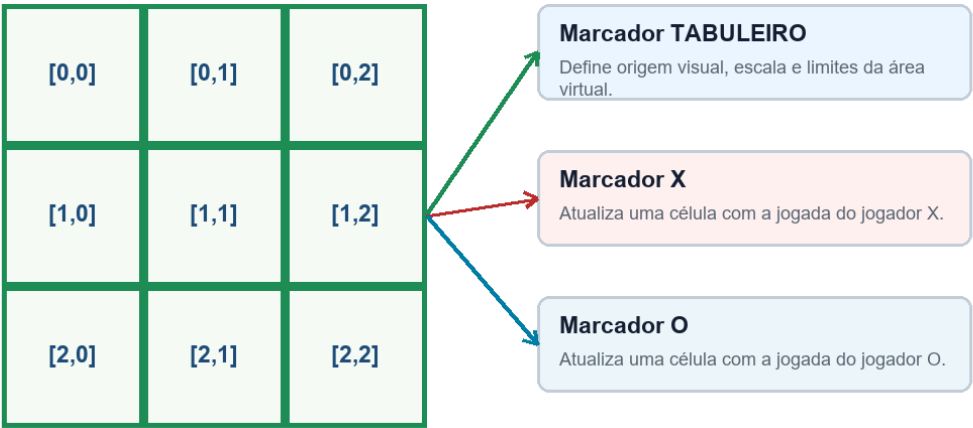
- 1. Inicialização:** o sistema lista câmeras disponíveis, recebe a escolha do usuário e abre o dispositivo selecionado.
- 2. Captura:** a aplicação lê frames continuamente, formando a base visual sobre a qual os objetos virtuais serão desenhados.
- 3. Deteção:** o QRCodeDetector localiza múltiplos marcadores no mesmo frame e retorna os textos decodificados e os vértices de cada QR Code.

- 4. **Classificação:** cada marcador é tratado conforme seu conteúdo. TABULEIRO gera a grade; X gera duas diagonais; O gera um círculo.
- 5. **Cálculo geométrico:** o sistema calcula centro, largura, altura e escala dos marcadores para manter proporção, rotação aparente e alinhamento visual.
- 6. **Renderização e exibição:** os overlays são rasterizados no próprio frame, exibidos em uma janela e atualizados até que o usuário pressione ESC.

4. Diagramas e Modelagem

4.1 Modelagem do Tabuleiro

O tabuleiro é modelado como uma matriz de estados 3 x 3. Cada célula pode assumir os valores vazio, X ou O. A etapa seguinte do projeto deve associar a posição detectada dos marcadores X e O às células da matriz, permitindo validação de jogadas e identificação de vitória, derrota ou empate.



Estado: matriz 3 x 3 com valores vazio, X ou O.

4.2 Estrutura de Dados Proposta

Elemento	Representação	Finalidade
Frame	Matriz de pixels BGR	Imagem capturada da câmera e modificada pelos overlays.
Marcador detectado	Texto + quatro pontos 2D	Identifica o tipo do QR Code e sua localização na imagem.
Tabuleiro virtual	Quatro pontos externos + duas divisões internas por eixo	Define a área gráfica onde a partida acontece.
Estado do jogo	Matriz 3 x 3	Registra posições ocupadas por X e O.
Jogada	tipo da peça + célula estimada	Atualiza a matriz quando uma peça é posicionada em uma célula válida.

4.3 Regras Lógicas Previstas

- A jogada só será aceita se a célula correspondente estiver vazia.
- A alternância entre X e O poderá ser controlada por turno, evitando duas jogadas seguidas do mesmo jogador.
- A verificação de vitória deverá testar três linhas, três colunas e duas diagonais.
- O empate será detectado quando as nove células estiverem preenchidas sem combinação vencedora.
- O HUD poderá exibir jogador atual, vencedor, empate e mensagens de erro quando uma jogada for inválida.

5. Conceitos Teóricos Envolvidos

Realidade Aumentada e Programação em Python

A Realidade Aumentada vem sendo cada vez mais utilizada junto à programação para criar experiências interativas e imersivas. Dentro desse contexto, a linguagem Python se destaca por ser simples, versátil e possuir bibliotecas voltadas para visão computacional e desenvolvimento gráfico, como OpenCV e PyOpenGL. Isso faz com que muitos projetos acadêmicos utilizem Python no desenvolvimento de aplicações de realidade aumentada, principalmente na área educacional.

Segundo o trabalho “Além do Código: Potencializando o Ensino de Programação com Realidade Virtual e Aumentada”, a utilização dessas tecnologias torna o aprendizado de programação mais dinâmico e próximo da realidade dos estudantes. O estudo destaca que a realidade aumentada pode facilitar o entendimento de algoritmos e lógica de programação, permitindo uma interação mais prática e visual durante o aprendizado.

Outro estudo relacionado ao ensino de Python mostra a aplicação da realidade aumentada no ensino da estrutura “for range”, muito utilizada na linguagem Python. Os autores explicam que o uso de elementos virtuais e animações ajuda os alunos a compreenderem melhor conceitos abstratos da programação, tornando as aulas mais atrativas e intuitivas.

Além da educação, a realidade aumentada programada em Python também pode ser aplicada em jogos, medicina, engenharia e treinamentos profissionais. Isso demonstra como a tecnologia vem crescendo e ganhando espaço em diferentes áreas, principalmente devido à evolução das bibliotecas de visão computacional e inteligência artificial integradas ao Python.

Realidade Aumentada Baseada em Vídeo (*Video-Based AR*)

A aplicação combina o fluxo de vídeo real capturado pelo sensor óptico com elementos virtuais gerados por computador, renderizados de forma síncrona no mesmo frame. O correto

alinhamento visuo-espacial depende estritamente da acurácia no rastreamento dos marcadores e do cálculo iterativo de *overlays* para garantir a ilusão de coexistência no espaço bidimensional.

Marcadores Fiduciais

Os QR Codes são empregados como sistemas de coordenadas locais conhecidos. Além de encapsularem strings identificadoras exclusivas (metadados), a extração geométrica de seus quatro vértices fornece as referências espaciais fundamentais para estimar a translação, escala e orientação aparente do objeto em relação ao plano de projeção da imagem.

Processamento de Imagens em Tempo Real

Cada frame do fluxo de captura é tratado como uma matriz multidimensional de pixels ($M \times N \times C$). O pipeline do sistema exige algoritmos eficientes de filtragem, binarização e segmentação geométrica locais para computar coordenadas e renderizar primitivas dentro do orçamento de tempo do laço principal, evitando latências perceptíveis.

Transformações Geométricas e Projeções

A partir do arranjo planar dos vértices do marcador, o sistema infere propriedades geométricas como ponto central, largura, altura e fator de escala por meio do cálculo de normas vetoriais euclidianas. Essa abordagem fundamenta-se em transformações de matrizes bidimensionais, mapeando translações, rotações e distorções de perspectiva (homografia 2D implícita) para interpolar os quadrantes expandidos do tabuleiro.

Pipeline de Renderização e Rasterização 2D

Os elementos gráficos do jogo (peças e linhas) são descritos como primitivas vetoriais matemáticas. Através de algoritmos de rasterização da biblioteca OpenCV (como as funções estruturais *line*, *circle* e *polylines*), essas descrições geométricas abstratas são discretizadas e convertidas em arranjos de pixels coloridos sobrepostos diretamente na matriz de imagem de fundo.

Modelagem Lógica e Abstração de Estados

O desacoplamento através de uma matriz algébrica 3×3 separa estritamente a representação visual da lógica de negócios. Esse padrão arquitetural garante que a interface baseada em Realidade Aumentada atue estritamente como um dispositivo de visualização e captura de entrada de dados, delegando a validação de regras de integridade e transição de turnos a um motor de estado interno abstrato.

6. Limitações Identificadas e Decisões Técnicas

- A versão atual usa sobreposição 2D, suficiente para a etapa atual, mas ainda não estima pose 3D completa da câmera.
- O uso de QR Code simplifica a identificação dos marcadores, porque o texto decodificado diferencia TABULEIRO, X e O.
- A estabilidade do rastreamento depende de iluminação, foco da câmera, tamanho do QR Code e oclusões durante a captura.
- A lógica de célula ainda precisa ser integrada ao marcador TABULEIRO expandido, calculando em qual quadrante cada peça foi posicionada.
- Para as próximas etapas, o projeto deve priorizar validação de jogadas, detecção automática de vitória, feedback visual e melhoria da experiência de uso.

7. Planejamento para a Próxima Entrega

Atividade	Resultado esperado	Critério de validação
Mapear peças para células	Identificar em qual posição da matriz 3 x 3 cada QR Code X/O foi colocado.	Ao mover uma peça para cada célula, o sistema registra a posição correta.
Implementar regras do jogo	Detectar vitória, empate e jogadas inválidas.	Testes com combinações horizontais, verticais, diagonais e tabuleiro cheio.
Adicionar HUD	Exibir turno atual, mensagens e resultado da partida.	Usuário consegue entender o estado da partida pela tela.
Refinar estabilidade	Reduzir tremulação e melhorar tolerância a ruído visual.	Marcadores permanecem rastreados em condições reais de uso.

8. Referências

AZUMA, Ronald T. A Survey of Augmented Reality. Presence: Teleoperators and Virtual Environments, v. 6, n. 4, p. 355-385, 1997. Disponível em: <https://direct.mit.edu/pvar/article-pdf/6/4/355/1623026/pres.1997.6.4.355.pdf>

FARIAS, M. F. C. Ensino do “for range” em Python com realidade aumentada. Universidade Federal da Paraíba, 2016. Disponível em: <https://repositorio.ufpb.br/jspui/bitstream/123456789/3325/1/MFC01122016.pdf>. Acesso em: 27 maio 2026.

OPENCV. QRCodeDetector Class Reference. Documentação oficial do OpenCV. Disponível em: https://docs.opencv.org/3.4/de/dc3/classcv_1_1QRCodeDetector.html

OPENCV. Drawing Functions in OpenCV. Documentação oficial do OpenCV. Disponível em: https://docs.opencv.org/4.x/dc/da5/tutorial_py_drawing_functions.html

OPENCV. Geometric Transformations of Images. Documentação oficial do OpenCV. Disponível em: https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html

MIGUEL, P. D. X. Além do código: potencializando o ensino de programação com realidade virtual e aumentada. Instituto Federal de Educação, Ciência e Tecnologia do Sertão Pernambucano, 2023. Disponível em: https://releia.ifsertaope.edu.br/jspui/bitstream/123456789/1183/1/ALEM_DO_CODIGO_PO_TENCIALIZANDO_O_ENSINO_DE_PROGRAMACAO_COM_REALIDADE_VIRTUAL_E_AUMENTADA.pdf. Acesso em: 27 maio 2026.

NUMPY. numpy.linalg.norm. Documentação oficial do NumPy. Disponível em: <https://numpy.org/doc/2.0/reference/generated/numpy.linalg.norm.html>

SZELISKI, Richard. Computer Vision: Algorithms and Applications. 2. ed. Springer, 2022. Disponível em: <https://szeliski.org/Book/>